Enter Docker Environment (For Raspberry Pi Only)

0. Applicable Models Pre-Introduction

> **Important**
>
> - 🟢 **Users who MUST read this tutorial**: **Raspberry Pi 5 (Raspberry Pi 5)** mainboard users. To ensure system stability and isolation, the ROS 2 Humble chassis driver, radar driver, mapping navigation, and AI algorithms for the Raspberry Pi car are all deployed in a **Docker container** (fixed container name: `a1pro_dev`).
> - ⚪ **Users who should skip this section directly**: **RDK X5**, **Jetson Orin (Nano / NX)** mainboard users. These mainboards come with native ROS 2 development environment pre-installed, all commands can be executed directly in the mainboard terminal, **no Docker needed**!

---

1. Core Concepts Simplified

Before starting, you only need to understand the collaboration relationship between the host and container:

Concept	Simple Understanding	Default Terminal Feature	Responsibilities
Host	Raspberry Pi's own operating system	`pi@raspberrypi:~$`	Hardware power management, network Wi-Fi, desktop display management, Docker daemon
Docker Container	Pre-installed ROS 2 and all algorithm development sandbox	`root@raspberrypi:~#`	**All ROS 2 driver startup, radar mapping, autonomous navigation, robotic arm and vision cases must be run here**

> **⚠ Warning**
>
> **Newbie Pitfall Prevention First Law**:
>
> Many beginners open a new terminal and directly type `ros2 ...` on the host (`pi@raspberrypi:~$`), the system will prompt `command not found: ros2`.
> **Please remember: Before executing any ROS 2 related compilation, node startup, or cases, you must first enter the Docker container (terminal prompt is `root@...`)!**

---

2. Daily Most Frequently Used: One-Click Ultra-Fast Enter Container

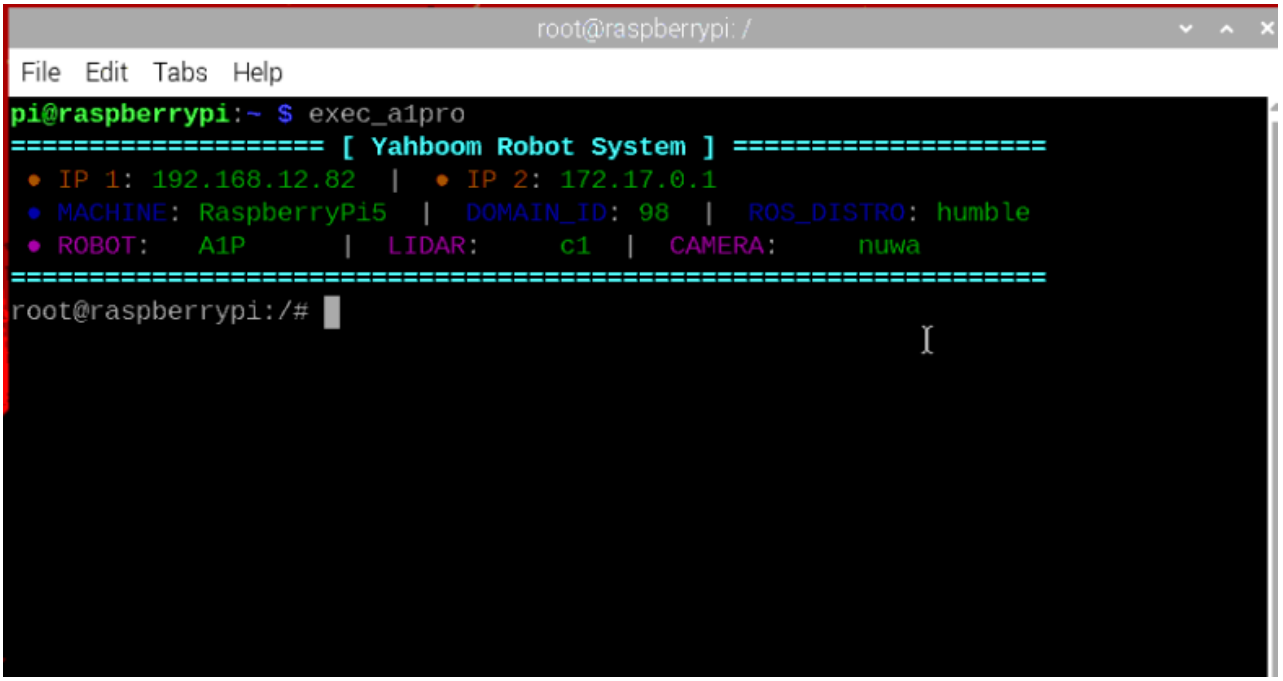
The system has pre-configured an ultra-simple shortcut command for you, daily entry to car development environment requires only one command:

Enter in the Raspberry Pi host terminal:

```
exec_a1pro
```

(Or in any terminal, enter standard command: `docker exec -it a1pro_dev bash`*)*

The terminal will immediately and seamlessly connect to the Docker container pre-installed with all car drivers and ROS 2 Humble environment:



After entering, the terminal will display the current car's basic status dashboard (IP address, car model, radar type, camera type, and Domain ID). At this point, you can directly enter any ROS 2 command to start experiments!

---

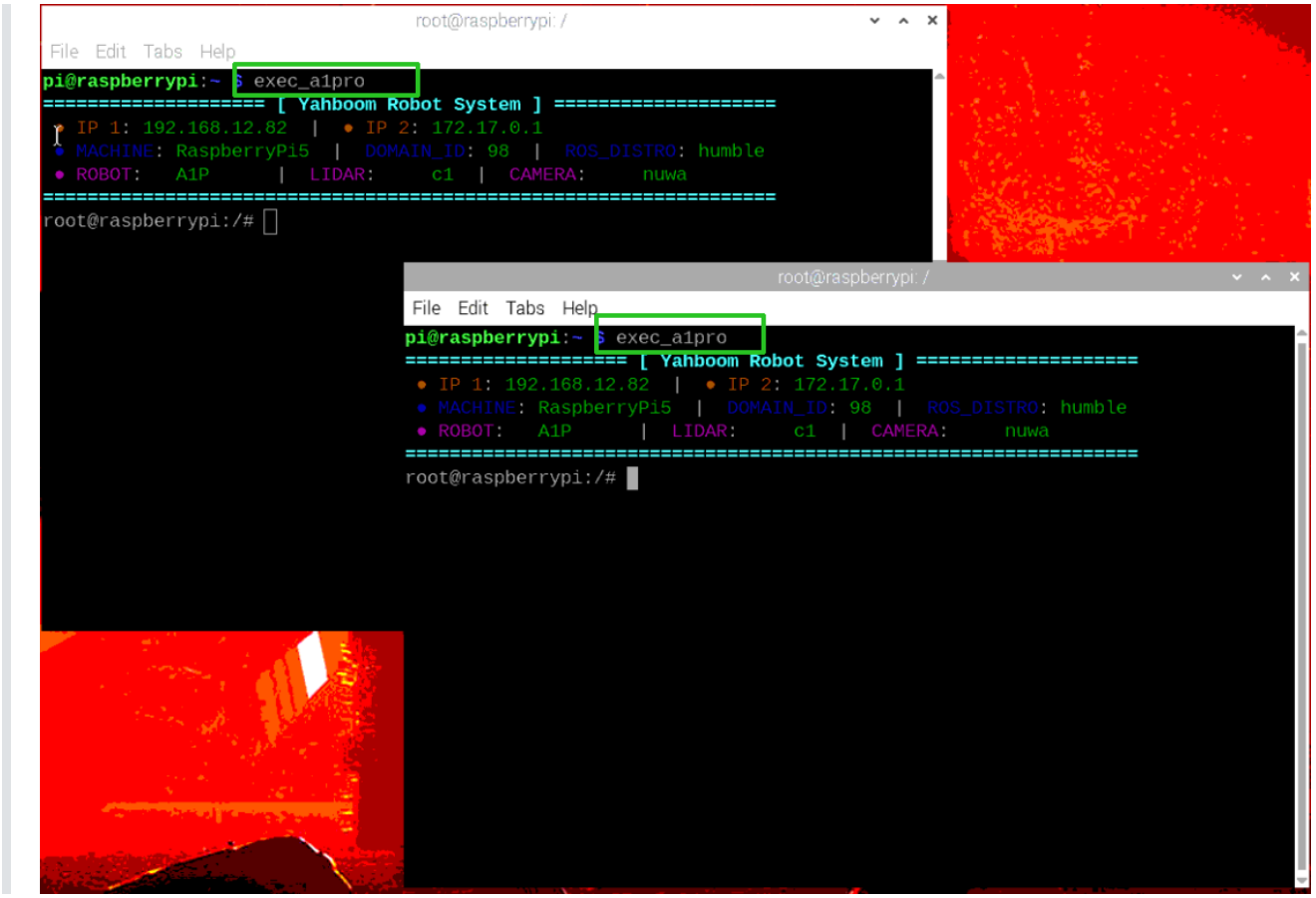
3. Multi-Terminal Collaborative Operation (Open Multiple Windows for Experiments)

In actual robot development, you usually need to **open multiple terminal windows simultaneously** to work together (for example: window 1 starts chassis radar driver, window 2 starts keyboard control, window 3 views RViz interface):

Since each new terminal window on Raspberry Pi defaults to the host environment (`pi@...`), you only need to **enter** `exec_a1pro` **in each new terminal window**:

```
exec_a1pro
```

This allows that window to also enter the same running Docker container, windows are independent from each other and topics are interconnected.



4. Container Auto-Start and Underlying Service Management (Docker Compose)

The car uses industry-standard **Docker Compose** for unified orchestration and management. The car startup configuration file is located at `/home/pi/yahboomcar_ros2_ws/docker-compose.yml`.

1. Auto-Start on Boot (No Manual Start Required)

The container has been configured with `restart: unless-stopped` protection policy:

- **After Raspberry Pi is powered on and boots up, Docker will automatically pull up and keep the `a1pro_dev` container running in the background**.
- You can directly open terminal and type `exec_a1pro` to start using after boot, usually no need to manually type startup command.

2. Manual Wake-Up Container Shortcut Command

If you accidentally stop the container, you can directly type the system-level shortcut command in the host terminal to quickly wake up and open graphics permissions: **(Note: You need to start this command in the VNC interface for it to work)**

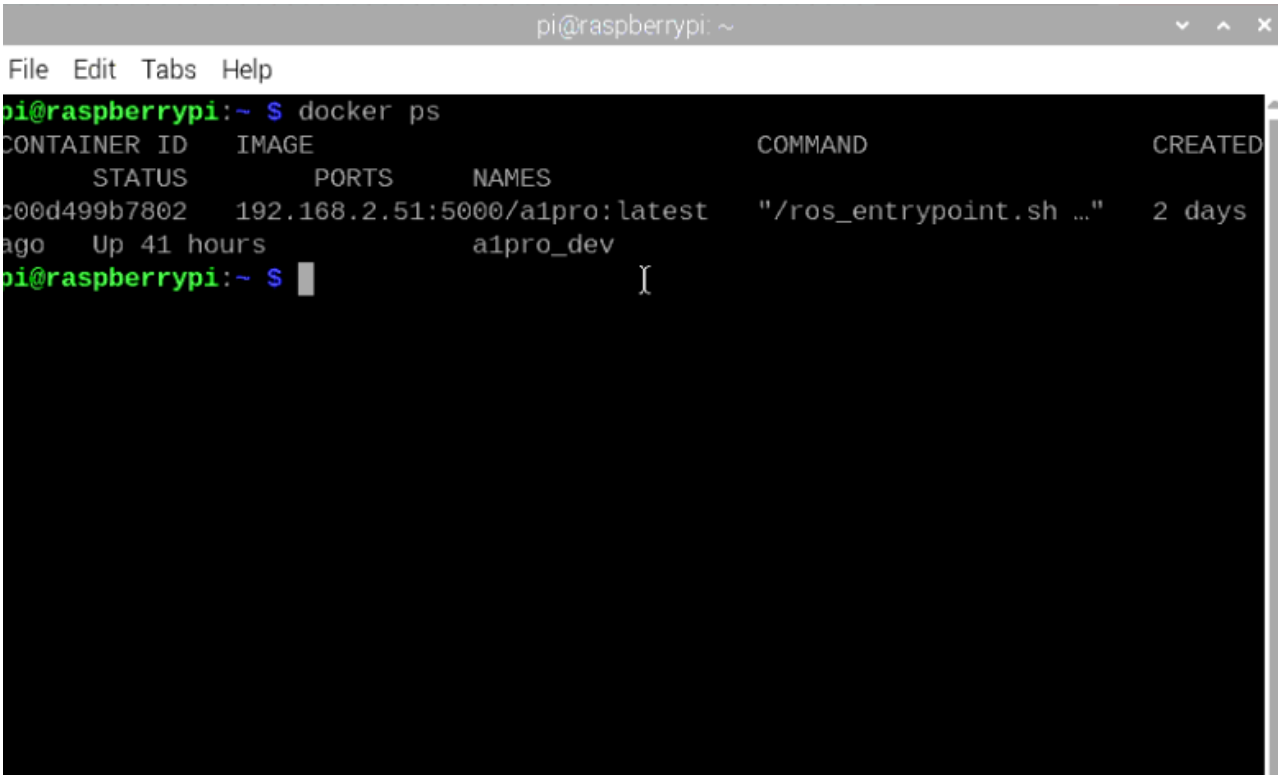
```
bringup_a1pro
```

(This command will automatically execute `xhost +` to open display permissions and enter the project directory to start the container)

3. View Container Current Running Status

Enter in the host terminal:

```
docker ps
```



If the status bar shows `Up ...` and the name is `a1pro_dev`, the container is running healthily.

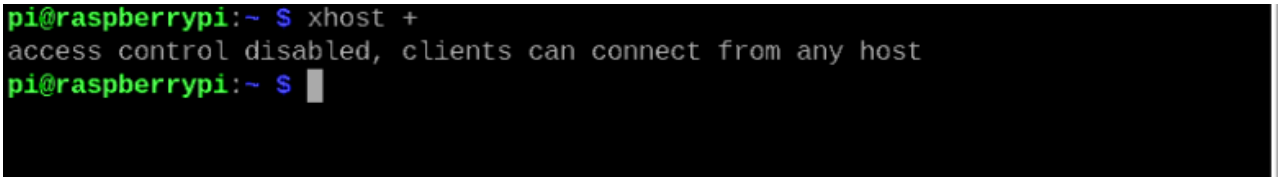
---

5. Graphical Interface Display (RViz2 / Camera Feed / GUI Applications)

When running programs that need to pop up graphical windows inside the Docker container (such as RViz2 to view laser point clouds, open camera feed windows, or run joystick control applications):

The `docker-compose.yml` has automatically configured X11 mounting and Chinese UTF-8 language environment (`LANG=zh_CN.UTF-8`), not only **windows can pop out to the desktop normally, but also the Chinese interface is clear without garbled characters**.

> **Tip**
>
> If you occasionally encounter graphical windows not popping up or error `cannot open display`, just type the following command in the **host desktop terminal** to open display permissions:
>
> ```
> xhost +
> ```
>
> Terminal output `access control disabled, clients can connect from any host` means the graphics display channel is unblocked.



6. Code and File Persistence Mechanism (Code Will Never Be Lost)

> **Note**
>
> **Why is code written in the container and saved maps absolutely safe after restart or shutdown?**
>
> This image is configured with bidirectional volume mapping (Volume Binding):
>
> - Host `/home/pi/yahboomcar_ros2_ws` ⟺ Container `/root/yahboomcar_ros2_ws`
> - Host `/home/pi/map` ⟺ Container `/root/map`
> - Host `/home/pi/MODELS` ⟺ Container `/root/MODELS`
>
> Any code modifications, new package creation, code compilation, or navigation map saves inside the Docker container **will be written to the Raspberry Pi host's hard drive in real-time**. Even if Docker container is recreated in the future (such as executing `docker compose down` for rebuild), your code and map data will still be safe!

7. Advanced Skill: How to Use `docker commit` to Solidify Backup Image After Modifying Container Environment

In daily advanced development of the car, you may perform some **system-level environment installation and changes** inside the container, for example:

- Installed new system tools (such as `lsof`, `htop`, `tree` or underlying dependency libraries) using `apt-get install` inside the container;
- Installed new Python algorithm libraries using `pip install` inside the container;
- Installed new services and global tools (such as `openclaw`) using `npm install -g` inside the container;
- Modified `/root/.bashrc` or system-level core configuration files inside the container.

> **Important**
>
> **Important Mechanism Distinction**:
>
> - `/home/pi/yahboomcar_ros2_ws`, `map`, `MODELS` are **data volume mounted directories**, changes are saved to Raspberry Pi hard drive in real-time, **no commit needed and will never be lost**!
> - However, environment changes installed in system root directories (`/usr`, `/lib`, `/etc`) via `apt`, `pip`, `npm` will be lost if not solidified into the underlying image when the container is recreated (such as executing `docker compose down` and rebuilding).

> Therefore, **when you have installed new environment inside the container and confirmed it works normally, it is recommended to execute `docker commit` on the host to solidify and backup the image!**

Step 1: Exit Container to Return to Raspberry Pi Host Terminal

If currently inside the container, first enter `exit` to return to host terminal (`pi@raspberrypi:~$`).

Step 2: Execute Overwrite Commit Solidification Command

Enter the following command in the host terminal:

```
docker commit a1pro_dev 192.168.2.51:5000/a1pro:latest
```

After execution, the terminal will output a sha256 image ID composed of numbers and letters (takes from a few seconds to over ten seconds, depending on change size), indicating the current container's environment has been completely solidified and stored in the local image:



> **Tip**
>
> **Master Anti-Rollover Technique: Make a Dated Safe Snapshot Backup**:
>
> If you are about to perform high-risk system library modifications, you can snapshot the image with today's date before modifying, leaving a way to regret:
>
> ```
> docker commit a1pro_dev 192.168.2.51:5000/a1pro:backup_$(date +%Y%m%d)
> ```
>
> This way, even if you accidentally break the environment in subsequent operations, you can always switch back to the snapshot image on that date.

---

8. Exiting Current Terminal

When an experiment is completed and that terminal is no longer needed, enter inside the container:

```
exit
```

After pressing Enter, you can exit that terminal and return to the host state.
Note: `exit` only exits this terminal window, the background `a1pro_dev` container and other car background services are still running smoothly!

---

9. Docker Common Operation Commands Quick Reference

Operation Need	Recommended Shortcut Command	Standard Docker / Compose Command	Execution Environment
One-click enter car development container	`exec_a1pro`	`docker exec -it a1pro_dev bash`	Host terminal
One-click wake up car container	`bringup_a1pro`	`cd ~/yahboomcar_ros2_ws && docker compose start`	Host terminal
Solidify/overwrite backup container environment	—	`docker commit a1pro_dev 192.168.2.51:5000/a1pro:latest`	Host terminal
View containers currently running	—	`docker ps`	Host terminal
Open desktop graphics display permissions	—	`xhost +`	Host desktop terminal
Completely restart car container	—	`cd ~/yahboomcar_ros2_ws && docker compose restart`	Host terminal
Recompile and pull up container	—	`cd ~/yahboomcar_ros2_ws && docker compose up -d`	Host terminal
Exit current terminal session	—	`exit`	Inside container terminal